

02차시 · LLM 구조와 동작 원리, 주요 모델 비교

02차시 · LLM 구조와 동작 원리, 주요 모델 비교

1일차 11:00–12:00 · 60분 · 10페이지
기준: 2026학년도 보수교육(융합) 과정안내서 / Ubuntu / Python 3.11 / VS Code
작성일: 2026-11-04

60분 운영

시간	활동	산출물
0–8분	언어 모델의 입력·출력 이해	토큰·문맥 개념 설명
8–18분	Transformer와 Attention 이해	블록도 읽기
18–28분	Decoder 구조·학습과 추론 구분	단계별 동작 설명
28–38분	모델·토크나이저 준비	모델 정보 확인
38–52분	토큰 수·생성 실습	명령 출력 기록
52–60분	비교·평가·다음 차시 연결	실습 체크리스트

목표: LLM(대규모 언어 모델)의 다음 토큰 예측 구조를 설명하고, 모델 크기·문맥 길이·라이선스·실행 자원 조건을 구별한다. 실제 실행은 선택된 장비에서 별도 검증한다.

이번 차시 선택 모델: `Qwen/Qwen2.5-0.5B-Instruct`. 원본 안내서가 특정 모델을 지정한 것은 아니다.

배경 · 선수지식 · LLM이 처리하는 정보

1. 입력 문장은 곧바로 숫자가 되지 않는다

`문자열 → 토크나이저 → 토큰 ID → 임베딩 벡터 → Transformer → 다음 토큰 확률 → 토큰 선택 → 문자열 복원`

토큰(Token, 텍스트 처리 단위)은 단어와 반드시 일치하지 않는다. 한글·숫자·기호·공백의 분리 방식은 모델의 토크나이저마다 다르다. 같은 문장도 서로 다른 모델에서는 토큰 수가 다를 수 있다.

임베딩(Embedding, 수치 벡터 표현)은 각 토큰을 학습 가능한 고차원 숫자 배열로 바꾼다. 단어의 사전적 의미를 고정된 숫자 하나로 표현하는 것이 아니라, 다른 토큰과 계산할 수 있는 형태로 바꾸는 과정이다.

문맥 길이(Context length)는 한 번의 처리에서 모델이 다룰 수 있는 토큰 범위다. 긴 문서라고 해서 모든 토큰이 무제한 들어가는 것은 아니다. 모델 카드의 문맥 길이와 실제 추론 설정을 모두 확인한다.

선수지식 점검

- Python 가상환경 활성화, `python --version` 확인
- `pip`로 라이브러리 설치와 `import`의 차이 이해
- 입력값과 출력값, 함수 호출, 배열 인덱스의 기본 개념

질문

`"안녕하세요"`가 반드시 한 개 토큰인가? **아니다.** 모델마다 다르며 실제 토크나이저로 확인해야 한다.

핵심 원리 · 다음 토큰 예측과 확률

1. 언어 모델의 기본 계산

문맥 $x_1, x_2, \ldots, x_t$가 주어졌을 때 다음 토큰 x_{t+1}의 조건부 확률을 예측한다.

$P(x_{t+1} \mid x_1, x_2, \ldots, x_t)$

예: `"오늘 날씨가"`라는 입력 뒤에는 여러 토큰 후보가 있다. 모델은 후보별 점수(Logits, 정규화 전 점수)를 계산하고, Softmax(소프트맥스, 점수를 확률로 바꾸는 함수)로 확률 분포를 얻는다.

Greedy decoding(그리디 디코딩, 매 단계 최고 확률 토큰 선택)은 `do_sample=False`로 실행할 수 있다. 같은 입력·모델·설정에서는 일반적으로 재현성이 높지만, 하드웨어·라이브러리 구현에 따른 미세 차이는 있을 수 있다.

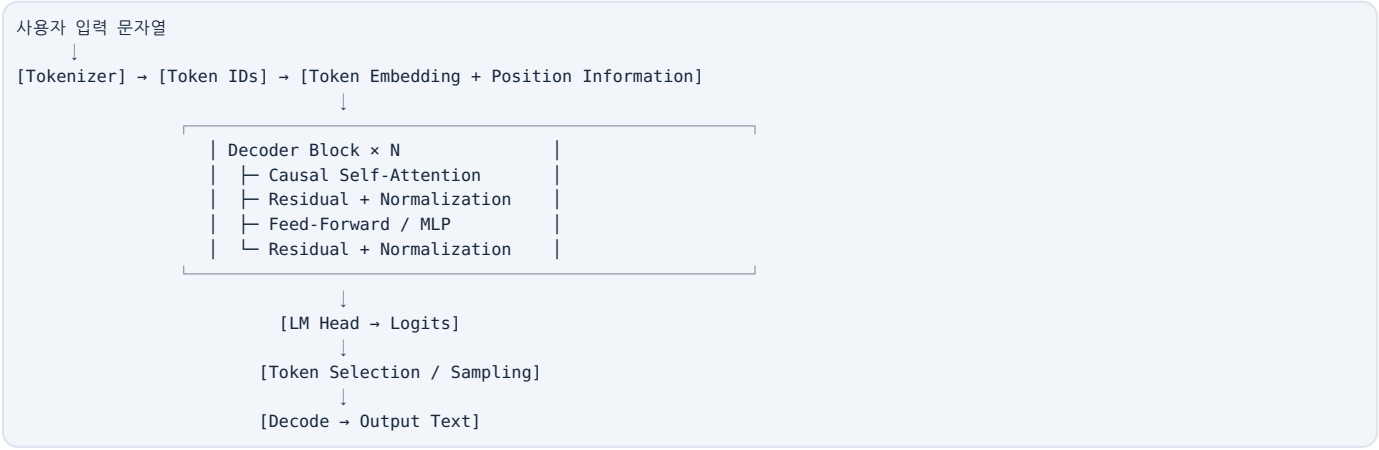
2. 학습과 추론은 다르다

구분	학습	추론
입력	학습 데이터	사용자 프롬프트
계산	순전파 + 손실 + 역전파	주로 순전파 + 토큰 생성
가중치	업데이트	고정
대표 출력	체크포인트·손실값	응답 문자열

착안: 4차시 LoRA는 전체 가중치를 모두 갱신하는 대신 소수의 학습 가능한 추가 행렬을 조정한다.

구조도 · Decoder 기반 Transformer

1. 전체 추론 흐름



2. 인과적 마스킹(Causal mask)

현재 위치의 토큰이 **미래 위치의 토큰을 보지 못하도록** Attention(어텐션, 문맥의 관련성에 가중치를 두는 연산)을 제한한다. 학습 때는 여러 위치를 병렬 계산할 수 있지만, 생성 시 새 토큰은 일반적으로 순차적으로 만들어진다.

3. Attention의 세 입력

`Q(Query, 질의)`, `K(Key, 키)`, `V(Value, 값)` 로 관련성을 계산한다. 핵심은 **현재 위치가 앞선 토큰의 어떤 정보를 얼마나 참고할지**를 결정하는 것이다.

주의: 그림은 교육용 일반화다. 실제 모델은 RMSNorm, RoPE, GQA 등 세부 구성이 다를 수 있다.

주요 모델 비교 · 무엇을 기준으로 고를까

1. 모델을 비교할 때의 축

항목	확인할 내용	실습 영향
파라미터 수	0.5B, 1.5B, 7B 등	저장공간·메모리·연산량
학습 목적	Base / Instruct	대화 응답 적합성
문맥 길이	모델 카드·설정값	긴 프롬프트 처리
토크나이저	어휘·채팅 템플릿	토큰 수·출력 형식
정밀도	FP32/FP16/BF16/INT4	메모리·지원 장치
라이선스	모델별 조건	재배포·상업적 사용

후보 예시: Qwen2.5 0.5B Instruct(소형 실습), Qwen2.5 1.5B Instruct(중간 규모), Llama 계열 및 Gemma 계열(모델별 사용 조건 확인). 특정 계열의 모든 버전이 동일한 라이선스·문맥 길이·하드웨어 요구사항을 갖는 것은 아니다.

2. 메모리 추정의 한계

파라미터 가중치만 고려한 이론적 크기는 `파라미터 수 × 저장 바이트`다. 0.5B 모델의 FP16 가중치만 단순 계산하면 약 1GB지만, **실제 실행에는 임시 텐서·KV 캐시·프레임워크 오버헤드가 추가**된다. 4비트 양자화가 모든 메모리 사용을 1/4로 만드는 것도 아니다.

3. 선택 기준

교육용 첫 실행은 작은 모델을 사용하고, 다음 차시에서 Ollama 및 Hugging Face의 실행 경로를 비교한다.

준비 · 설치 · 실행 파일 구성

1. 폴더 구조

```
lab_llm/
├── .venv/                  # 01차시에서 만든 가상환경
├── lesson02_infer.py       # 이번 차시 실습
└── docs/
    └── lesson02_result.txt # 실행 결과 기록
```

2. Ubuntu 터미널 절차

```
cd ~/lab_llm                     # 실습 프로젝트로 이동
source .venv/bin/activate        # 가상환경 활성화
python --version                 # Python 3.11인지 확인
python -m pip show transformers torch  # 패키지 설치 상태 확인
mkdir -p docs                   # 실행 결과 저장 폴더 준비
```

패키지가 없으면 01차시의 설치 기준에 따라 설치한다. 이미 정상 설치되어 있다면 불필요하게 재설치하지 않는다.

3. 사전 확인

최초 실행에는 모델 파일 다운로드를 위한 네트워크와 디스크 공간이 필요하다.

CPU 실행도 가능하나 장비에 따라 시간이 오래 걸릴 수 있다.

본 예제는 기본적으로 CPU에서 실행되도록 작성했다. GPU 사용을 전제로 성능 수치를 약속하지 않는다.

원격 저장소의 코드를 실행하도록 허용하는 `trust_remote_code=True`는 이 예제에서 사용하지 않는다.

4. 실행 명령

```
python lesson02_infer.py --max-new-tokens 80
```

예상 관찰: 입력 토큰 수, 생성 토큰 수, 모델 응답이 순서대로 출력된다. 숫자·문장은 실제 실행 때 달라질 수 있다.

예제 코드 · 토큰화와 문장 생성

1. 실습 파일: `lesson02_infer.py`

```
# lesson02_infer.py — 02차시 실습: 토큰화와 짧은 문장 생성
import argparse
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"  # 교육용 선택 모델


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--prompt", default="FPGA가 무엇인지 두 문장으로 설명해 주세요.")
    parser.add_argument("--max-new-tokens", type=int, default=80)
    args = parser.parse_args()

   # 모델과 같은 저장소의 토크나이저를 사용한다.
    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
    model = AutoModelForCausalLM.from_pretrained(MODEL_ID)
    model.eval()  # 평가/추론 모드: 학습용 동작 비활성화

    messages = [{"role": "user", "content": args.prompt}]
   # 채팅 템플릿이 사용자/어시스턴트 구분을 모델 형식에 맞게 변환한다.
    encoded = tokenizer.apply_chat_template(
        messages, tokenize=True, add_generation_prompt=True,
        return_tensors="pt"
   )
    with torch.inference_mode():  # 역전파용 계산 그래프를 만들지 않는다.
        output = model.generate(
            encoded, max_new_tokens=args.max_new_tokens,
            do_sample=False, pad_token_id=tokenizer.eos_token_id
       )
    new_ids = output[0, encoded.shape[-1]:]  # 입력 토큰을 제외한 생성 부분
    print("입력 토큰 수:", encoded.shape[-1])
    print("생성 토큰 수:", new_ids.shape[-1])
    print("모델 응답:", tokenizer.decode(new_ids, skip_special_tokens=True))


if __name__ == "__main__":
    main()
```

핵심: 채팅 모델의 프롬프트는 단순 문자열 연결이 아니라 해당 모델의 채팅 템플릿을 사용한다. 출력 전체를 그대로 디코딩하면 입력 프롬프트까지 함께 출력될 수 있으므로 입력 길이 이후의 토큰만 선택한다.

코드 해설 · 입력·출력 검증

1. 주요 코드와 의미

코드	의미	검증 방법
`AutoTokenizer.from_pretrained()`	모델에 맞는 토크나이저 로드	다운로드·로딩 오류 확인
`AutoModelForCausalLM.from_pretrained()`	인과 언어 모델 가중치 로드	모델 로딩 완료 확인
`apply_chat_template(...)`	대화 형식 입력을 토큰 ID로 변환	입력 토큰 수 확인
`torch.inference_mode()`	추론 중 자동 미분 기록 방지	역전파 미실행 확인
`model.generate(...)`	다음 토큰을 반복 생성	생성 토큰 수 확인
`output[0, encoded.shape[-1]:]`	입력 부분을 제외한 출력 추출	응답에 프롬프트가 중복되지 않는지 확인

2. 실행 결과 예시(실제 실행값 아님)

```
입력 토큰 수: (실행 환경에서 확인)
생성 토큰 수: (실행 환경에서 확인)
모델 응답: (실제 생성된 문장)
```

3. 흔한 오해

토큰 수와 글자 수는 같지 않다.

모델 크기가 크면 언제나 모든 작업에서 성능이 좋은 것은 아니다.

`do_sample=False`는 출력 선택 방식을 정하는 옵션이지 학습 여부를 결정하는 옵션이 아니다.

`model.eval()`은 모델의 평가 동작을 설정하지만, 단독으로 자동 미분을 끄지는 않는다.

확인 질문: `encoded.shape[-1]`은 무엇을 뜻하는가? → 입력 시퀀스의 토큰 길이.

따라하기 실습 · 프롬프트 비교와 오류 대응

실습 A: 같은 모델, 다른 프롬프트

```
python lesson02_infer.py --prompt "FPGA를 한 문장으로 설명해 주세요." --max-new-tokens 60
python lesson02_infer.py --prompt "FPGA와 CPU의 차이를 표 없이 설명해 주세요." --max-new-tokens 100
```

두 실행의 **입력 토큰 수, 생성 토큰 수, 응답 형식**을 비교한다. 길이가 다른 프롬프트는 출력 내용과 연산 시간에 영향을 줄 수 있다.

실습 B: 출력 길이 제한

```
python lesson02_infer.py --prompt "UART의 역할을 설명해 주세요." --max-new-tokens 20
python lesson02_infer.py --prompt "UART의 역할을 설명해 주세요." --max-new-tokens 80
```

`max_new_tokens`는 새로 생성하는 토큰 수의 상한이다. 반드시 정확히 해당 개수만큼 생성되는 것은 아니다. 종료 토큰을 만나면 일찍 멈출 수 있다.

오류 점검

현상	원인 후보	우선 조치
`ModuleNotFoundError`	가상환경/패키지	`.venv` 활성화, `pip show`
다운로드 실패	네트워크·접근 제한	모델 저장소 접근 확인
메모리 부족	가중치·KV 캐시	다른 프로세스 종료, 더 작은 모델
출력이 느림	CPU 추론·긴 출력	`max-new-tokens` 축소
채팅 템플릿 오류	모델·토크나이저 불일치	동일 저장소 ID 사용

확장: 프롬프트를 3개로 늘리고 같은 설정에서 결과를 비교한다. 실측 시간이 없다면 속도 비교 결과를 추정으로 쓰지 않는다.

평가 · 핵심 정리 · 참고자료

1. 실습 제출 체크리스트

[ ] Python 3.11 가상환경을 확인했다.

[ ] 토크나이저와 모델을 같은 저장소에서 불러왔다.

[ ] 입력 토큰 수와 생성 토큰 수를 기록했다.

[ ] `max_new_tokens`를 바꾸어 출력 차이를 확인했다.

[ ] 추론과 학습에서 가중치 갱신 여부가 다름을 설명할 수 있다.

[ ] 모델 비교에 파라미터·문맥 길이·라이선스를 포함했다.

2. 확인 문제

1. 토큰과 글자 수가 다른 이유는? → 토크나이저의 분할 규칙이 다르기 때문이다.
2. Decoder의 causal mask는 왜 필요한가? → 미래 토큰 정보 누출을 방지한다.
3. `do_sample=False`의 역할은? → 확률적 샘플링 대신 결정적 선택 방식을 사용한다.
4. 모델의 파라미터 크기만으로 VRAM 요구량을 확정할 수 있는가? → 아니다.
5. 다음 차시에서 비교할 실행 경로는? → Ollama와 Hugging Face.

3. 핵심 정리

`입력 문자열 → 토큰 → 임베딩 → Decoder Transformer → 다음 토큰 선택 → 응답`의 전체 흐름을 설명할 수 있어야 한다. 모델 선택은 성능뿐 아니라 **메모리, 실행 환경, 토크나이저, 라이선스**를 함께 고려한다.

4. 공식 참고자료

Hugging Face Transformers: https://huggingface.co/docs/transformers/index

Hugging Face Chat templates: https://huggingface.co/docs/transformers/chat_templating

Hugging Face Text generation: https://huggingface.co/docs/transformers/main_classes/text_generation

Qwen2.5-0.5B-Instruct 모델 카드: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

Attention Is All You Need: https://arxiv.org/abs/1706.03762

자료 작성일: 2026-11-04. 위 공식 링크는 API·모델 조건을 재확인하기 위한 기준이며, 본 실행에서는 외부 사이트의 최신 본문을 실시간 대조하지 못했다. 실제 모델 다운로드·GPU 추론도 미실행이다.

적대적 검증·개선 기록 (2026-11-05)

반영: 기존 코드·교안의 핵심 구조를 보존했습니다. 실제 GPU 실험은 별도 실행 검증이 필요합니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)